

```

1 package myPackage;
2 import java.util.*;
3
4
5 public class Cat_Feeding {
6
7     private static final String DATA_FILE = "cat_data.txt"; //고양이 정보파일 경로
8
9     static class Cat implements Serializable {
10         private static final long serialVersionUID = 1L; //클래스 버전을 나타내는 ID
11
12         private String name;
13         private String breed;
14         private double weight;
15         private int activityLevel;
16         private double dailyFood;
17         private double feedingPortion;
18         private List<String> feedingTimes = new ArrayList<>();
19
20         public Cat(String name, String breed, double weight, int activityLevel) {
21             this.name = name;
22             this.breed = breed;
23             this.weight = weight;
24             this.activityLevel = activityLevel;
25         }
26
27         public String getName() { return name; }
28         public String getBreed() { return breed; }
29         public double getWeight() { return weight; }
30         public int getActivityLevel() { return activityLevel; }
31
32         public void setName(String name) { this.name = name; }
33         public void setBreed(String breed) { this.breed = breed; }
34         public void setWeight(double weight) { this.weight = weight; }
35         public void setActivityLevel(int activityLevel) { this.activityLevel = activityLevel; }
36
37         public double calculateFoodAmount() {
38             double bmr = 70 * Math.pow(weight, 0.75); //기초대사량 계산
39             double activityFactor = switch (activityLevel) {
40                 case 1 -> 1.2;
41                 case 2 -> 1.4;
42                 case 3 -> 1.6;
43                 default -> 1.0;
44             };
45             return bmr * activityFactor * 33/100;
46         }
47
48         public double getDailyFood() { return dailyFood; }
49         public void setDailyFood(double dailyFood) { this.dailyFood = dailyFood; }
50
51         public double getFeedingPortion() { return feedingPortion; }
52         public void setFeedingPortion(double feedingPortion) { this.feedingPortion = feedingPortion; }
53
54         public List<String> getFeedingTimes() { return feedingTimes; }
55         public void addFeedingTime(String time) { feedingTimes.add(time); }
56         public void clearFeedingTimes() { feedingTimes.clear(); }
57     }
58
59     public static void main(String[] args) {
60         Scanner scanner = new Scanner(System.in);
61         Cat cat = loadCatData(); //파일에서 고양이 정보 로드
62
63         if (cat == null) { //입력된 고양이 정보 유무 확인 후 없다면 입력받기
64             System.out.println("저장된 고양이 정보가 없습니다.");
65             System.out.println("고양이 정보를 입력해주세요.");
66             cat = addCatInfo(scanner);
67             saveCatData(cat); //파일에 저장
68         }
69
70

```

```

71         System.out.println("고양이 자동 급식기");
72
73         while(true) {
74             System.out.println("[메뉴]");
75             System.out.println("1. 고양이 정보 확인 및 수정");
76             System.out.println("2. 적정 사료량 계산 및 배식시간 예약");
77             System.out.println("3. 배식 실행");
78             System.out.println("4. 종료");
79             System.out.println("메뉴를 선택해주세요: ");
80             int choice = scanner.nextInt();
81
82             switch (choice) {
83                 case 1:
84                     CatInfo(cat, scanner);
85                     saveCatData(cat);
86                     break;
87                 case 2:
88                     calculateAndSchedule(cat, scanner);
89                     break;
90                 case 3:
91                     executeFeeding(cat);
92                     break;
93                 case 4:
94                     System.out.println("프로그램을 종료합니다");
95                     scanner.close();
96                     return;
97                 default :
98                     System.out.println("잘못된 입력입니다. ");
99             }
100         }
101     }
102
103     //0. 고양이 정보 입력 메서드
104     private static Cat addCatInfo(Scanner scanner) {
105         scanner.nextLine(); //입력 버퍼 비우기
106         System.out.print("고양이 이름: ");
107         String name = scanner.nextLine();
108         System.out.print("품종: ");
109         String breed = scanner.nextLine();
110         System.out.print("몸무게(kg, 소수점 두 자리까지): ");
111         double weight = scanner.nextInt();
112         System.out.print("활동수준(1: 낮음, 2: 보통, 3: 높음): ");
113         int activityLevel = scanner.nextInt();
114
115         return new Cat(name, breed, weight, activityLevel);
116     }
117

```

```

118 //1. 고양이 정보 확인 및 수정 메서드
119Ⓣ private static void CatInfo(Cat cat, Scanner scanner) {
120     System.out.println("\n고양이 정보");
121     System.out.println("1. 이름: " + cat.getName());
122     System.out.println("2. 품종: " + cat.getBreed());
123     System.out.println("3. 몸무게: " + cat.getWeight());
124     System.out.println("4. 활동수준: " + cat.getActivityLevel());
125     System.out.println("5. 수정하지 않고 상위메뉴로 돌아가기");
126     System.out.println("수정할 항목 번호를 입력하세요: ");
127     int choice = scanner.nextInt();
128
129     scanner.nextLine();
130     switch (choice) {
131     case 1:
132         System.out.print("새 이름: ");
133         cat.setName(scanner.nextLine());
134         break;
135     case 2:
136         System.out.print("새 품종: ");
137         cat.setBreed(scanner.nextLine());
138         break;
139     case 3:
140         System.out.print("새 몸무게(kg, 소수점 두 자리까지: ");
141         cat.setWeight(scanner.nextInt());
142         break;
143     case 4:
144         System.out.print("새 활동 수준(1: 낮음, 2: 보통, 3: 높음): ");
145         cat.setActivityLevel(scanner.nextInt());
146         break;
147     case 5:
148         System.out.print("수정을 취소합니다.");
149         return;
150     default:
151         System.out.println("잘못된 입력입니다.");
152     }
153     System.out.println("수정이 완료되었습니다.");
154 }

```



```

155 //2. 적정 사료량 계산 및 시간 예약 메서드
156 private static void calculateAndSchedule(Cat cat, Scanner scanner) {
157     double foodAmount = cat.calculateFoodAmount(); //적정 사료량 계산
158     System.out.printf("\n%s(이)의 하루 적정 사료양: %.2fg\n", cat.getName(), foodAmount);
159
160     int feedings; //배식 횟수 정하기
161     while(true) {
162         System.out.print("하루 배식 횟수를 정해주세요(최대 4회): ");
163         feedings = scanner.nextInt();
164         if (feedings < 1 || feedings > 4) {
165             System.out.println("잘못된 입력입니다.");
166         }
167         else break;
168     }
169
170     scanner.nextLine();
171     cat.clearFeedingTimes(); // 기존예약 초기화
172     for (int i = 0; i < feedings; i++) {
173         System.out.printf("%d번째 배식 시간(1~24): ", i+1);
174         String time = scanner.nextLine();
175         cat.addFeedingTime(time);
176     }
177
178     double portion = foodAmount / feedings; //1회분 사료량 계산
179     cat.setDailyFood(foodAmount);
180     cat.setFeedingPortion(portion);
181     System.out.printf("하루 %d번 한 번에 %.2fg씩 배식합니다.\n", feedings, portion);
182     System.out.println("배식 시간 설정이 완료되었습니다.");
183 }
184
185 //(아두이노 명령 전송)
186 private static void sendToArduino(String command) {
187     try {
188         SerialPort port = SerialPort.getCommPort("COM3"); //포트 번호에 따라 바뀜
189         port.setComPortParameters(9600, 8, 1, 0); //Baud Rate 설정(통신속도 설정)
190         port.openPort();
191
192         port.getOutputStream().write((command + "\n").getBytes());
193         port.getOutputStream().flush();
194         port.closePort();
195     }
196 }
197 */
198 //3. 배식 실행 메서드
199 private static void executeFeeding(Cat cat) {
200     System.out.println("배식스케줄");
201     for (String time : cat.getFeedingTimes()) {
202         System.out.printf("- %s시에 %.2fg 배식 예정\n", time, cat.getFeedingPortion());
203     }
204     /* 아두이노 연동 예제
205     for (String time : cat.getFeedingTimes()) {
206         String command = "FEED" + cat.getName() + ":" + cat.getFeedingPortion();
207         sendToArduino(command);
208     }*/
209 }
210 //4. 고양이 정보 파일 읽어오는 메서드
211 private static Cat loadCatData() {
212     try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(DATA_FILE))) {
213         return (Cat) ois.readObject();
214     } catch (FileNotFoundException e) {
215         return null;
216     } catch (IOException | ClassNotFoundException e) {
217         e.printStackTrace();
218         return null;
219     }
220 }

```

```

221 //5. 고양이 정보 파일 저장 메서드
222 private static void saveCatData(Cat cat) {
223     try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(DATA_FILE))) {
224         oos.writeObject(cat);
225     } catch (IOException e) {
226         e.printStackTrace();
227     }
228 }
229
230 }

```

저장된 고양이 정보가 없습니다.
고양이 정보를 입력해주세요.

고양이 이름: **조롱**

품종: **러시안블루**

몸무게(kg, 소수점 두 자리까지): **3**

활동수준(1: 낮음, 2: 보통, 3: 높음): **2 2**

고양이 자동 급식기

【메뉴】

1. 고양이 정보 확인 및 수정
2. 적정 사료량 계산 및 배식시간 예약
3. 배식 실행
4. 종료

메뉴를 선택해주세요:

1

고양이 정보

1. 이름: **조롱**
2. 품종: **러시안블루**
3. 몸무게: **3.0**
4. 활동수준: **2**
5. 수정하지 않고 상위메뉴로 돌아가기

수정할 항목 번호를 입력하세요:

4

새 활동 수준(1: 낮음, 2: 보통, 3: 높음)

수정이 완료되었습니다.

【메뉴】

1. 고양이 정보 확인 및 수정
2. 적정 사료량 계산 및 배식시간 예약
3. 배식 실행
4. 종료

메뉴를 선택해주세요:

조롱(이)의 하루 적정 사료량: **63.19g**

하루 배식 횟수를 정해주세요(최대 4회): **3**

1번째 배식 시간(1~24): **7**

2번째 배식 시간(1~24): **14**

3번째 배식 시간(1~24): **21**

하루 3번 한 번에 **21.06g**씩 배식합니다.

배식 시간 설정이 완료되었습니다.

【메뉴】

1. 고양이 정보 확인 및 수정
2. 적정 사료량 계산 및 배식시간 예약
3. 배식 실행
4. 종료

메뉴를 선택해주세요:

3

배식스케줄

- 7시에 **21.06g** 배식 예정

- 14시에 **21.06g** 배식 예정

- 21시에 **21.06g** 배식 예정

【메뉴】

1. 고양이 정보 확인 및 수정
2. 적정 사료량 계산 및 배식시간 예약
3. 배식 실행
4. 종료

메뉴를 선택해주세요: